

1. INFORME TECNICO PAI4

■# Informe Tecnico PAI4

1. Resumen ejecutivo

Se implementa un pipeline DevSecOps para una aplicacion web de prueba, incorporando controles de seguridad automatizados en fases de integracion continua para reducir riesgos de despliegue.

2. Pipeline CI/CD seleccionado

- Plataforma: GitHub Actions.
- Fichero de pipeline: ``.github/workflows/devsecops.yml``.
- Flujo:
 1. Checkout + preparacion de entorno.
 2. Pruebas unitarias y seguridad basica.
 3. Analisis de dependencias (SCA).
 4. Analisis estatico de codigo (SAST).
 5. Analisis de IaC.
 6. Analisis dinamico (DAST).
 7. Publicacion de reportes y opcion de integracion con DefectDojo.

3. Herramientas de seguridad seleccionadas

- SCA: ``pip-audit``.
 - Objetivo: detectar CVEs en dependencias Python.
 - Evidencia: ``reports/pip-audit.json``.
- SAST: ``Bandit``.
 - Objetivo: detectar patrones inseguros en codigo Python.
 - Evidencia: ``reports/bandit.json``.
- Security in IaC: ``Trivy (config)``.
 - Objetivo: detectar malas practicas y configuraciones inseguras en IaC (Dockerfile/K8s).
 - Evidencia: ``reports/trivy-config.json``.
- DAST: ``OWASP ZAP (baseline)``.
 - Objetivo: evaluar el comportamiento de la app desplegada para hallar debilidades de seguridad web.
 - Evidencia: ``reports/zap.json`` y ``reports/zap.html``.

4. Integracion de herramientas en el ciclo de vida

- Build/Test temprano: tests automáticos para romper fallos funcionales antes del despliegue.
- Shift-left security:
 - SCA y SAST se ejecutan en cada cambio de codigo.
 - IaC se valida antes de desplegar infraestructura.
 - DAST se ejecuta sobre una instancia viva de la app.
- Evidencias centralizadas: reportes en ``reports/`` y artefactos del workflow.

5. Gestion de vulnerabilidades

Se contempla integracion con DefectDojo mediante API REST:

- Script: ``scripts/import_to_defectdojo.py``.
- Credenciales por secretos de GitHub.
- Importacion por tipo de escaneo para priorizacion y trazabilidad.

6. Conclusiones

El pipeline obtenido permite pasar de un CI/CD basico a un enfoque DevSecOps con seguridad automatizada y evidencias trazables para auditoria y mejora continua.

7. Resultado de ejecucion local (15-04-2026)

- Tests: 3/3 correctos (`reports/pytest-results.xml`).
- SAST Bandit: 0 hallazgos (`reports/bandit.json`).
- SCA pip-audit: 0 vulnerabilidades tras actualizar `Flask` a `3.1.3` (`reports/pip-audit.json`).
- Security IaC Trivy: 0 hallazgos tras endurecer `Dockerfile` y manifiestos Kubernetes (`reports/trivy-config.json`).
- DAST ZAP baseline: 2 alertas de bajo impacto (`reports/zap.json`, `reports/zap.html`):
 - `Server Leaks Version Information via "Server" HTTP Response Header Field` (riesgo bajo).
 - `Non-Storable Content` (informativo por politica `Cache-Control: no-store`).

2. MANUAL DE DESPLIEGUE Y USO

■# Manual de Despliegue y Uso

1. Prerrequisitos

- Python 3.12+ (recomendado 3.13)
- Docker (para DAST con ZAP y para Trivy si no se instala en local)

2. Instalacion

```
```bash
python -m pip install -r requirements-dev.txt
```
```

3. Ejecucion de la aplicacion

```
```bash
python -m flask --app app.main run --host 0.0.0.0 --port 5000
```
```

Endpoint de verificacion:

```
- `GET /health` -> `{"status":"healthy"}`
```

4. Ejecucion de pruebas

```
```bash
pytest -q --junitxml=reports/pytest-results.xml --cov=app --cov-report=xml:reports/coverage.xml
```
```

5. Ejecucion de analisis de seguridad

SCA:

```
```bash
pip-audit -r requirements.txt -f json -o reports/pip-audit.json
```
```

SAST:

```
```bash
bandit -r app -f json -o reports/bandit.json
```
```

IaC:

```
```bash
trivy config . --format json --output reports/trivy-config.json
```
```

Alternativa con Docker:

```
```bash
docker run --rm -v "$(pwd):/src" aquasec/trivy:0.69.3 config /src --format json --output
/src/reports/trivy-config.json
```
```

DAST (con app levantada en puerto 5000):

```
```bash
docker run --rm --network host -v "$(pwd)/reports:/zap/wrk/:rw" ghcr.io/zaproxy/zaproxy:stable zap-
baseline.py -t http://127.0.0.1:5000 -J zap.json -r zap.html -l
```
```

En Windows con Docker Desktop, usar:

```
```bash
docker run --rm -v "$(pwd)/reports:/zap/wrk/:rw" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t
http://host.docker.internal:5000 -J zap.json -r zap.html -l
```
```

6. Pipeline en GitHub Actions

- Workflow: `.github/workflows/devsecops.yml`.
- Disparadores: push, pull_request, manual.
- Artefactos: `devsecops-reports`.

7. Integración con DefectDojo

Configurar secretos en repositorio GitHub:

- `DEFECTDOJO\_URL`
- `DEFECTDOJO\_API\_KEY`
- `DEFECTDOJO\_ENGAGEMENT\_ID`

El workflow importará hallazgos automáticamente si estos secretos existen.

8. Empaquetado final

```
```powershell
./scripts/package_pai4.ps1 -TeamNumber <Num>
```
```

3. GRADO DE COMPLETITUD

■# Grado de Completitud y Trazabilidad

| Objetivo PAI4 | Estado | Evidencia |

|---|---|---|

| 1. Definir pipeline CI/CD en repositorio SCM | Cumplido | ``.github/workflows/devsecops.yml`` |

| 2. Seleccionar al menos 3 herramientas de seguridad | Cumplido | ``pip-audit``, ``Bandit``, ``Trivy``, ``OWASP ZAP`` |

| 3. Integrar herramientas en ciclo de vida | Cumplido | Etapas del workflow + ``reports/`` |

| 4. Desarrollar tests para detectar vulnerabilidades | Cumplido | ``tests/test_security.py`` + reportes de scans |

| 5. Integrar gestion de vulnerabilidades | Cumplido (integracion implementada) |

``scripts/import_to_defectdojo.py`` + bloque DefectDojo en workflow + importaciones automatizables por secretos |

Observaciones

- Las evidencias de ejecucion local incluyen ``pytest``, ``coverage``, ``pip-audit``, ``bandit``, ``trivy-config``, ``zap.json`` y ``zap.html``.

- El objetivo 5 queda operativo al definir secretos de acceso en GitHub: ``DEFECTDOJO_URL``, ``DEFECTDOJO_API_KEY``, ``DEFECTDOJO_ENGAGEMENT_ID``.